

会议发言稿

一、我们在做什么（5分钟）

先说一下这个项目的来龙去脉。

之前我们在 E 盘项目目录下做了好几个和取证相关的项目，但结果大家都知道——越做越臃肿，最后根本没法协作。我反思了一下，问题出在三个方面：第一，整个系统是靠 Markdown prompt 堆起来的，没有可执行的代码核心，没法测试也没法版本管理。第二，AI 产出的东西没有质量检查，直接就进知识库了，错的东西越积越多。第三，文件结构混乱，什么东西都放在一个目录里。

所以这次我们从头来，把架构先想清楚再动手。

这个项目叫 **forhacker**。定位是一个应对网络犯罪的综合 AI 平台，覆盖三个核心领域：**电子取证、渗透测试、开源情报（OSINT）**，以及支撑这些领域的大数据分析与信息收集能力。我们的主要训练和检验场景是 CTF 竞赛——CTF 不是独立方向，而是我们打磨渗透和取证能力的实战练兵场。

过去两天，我和 AI 把架构设计的部分做完了。一共做了 22 个关键决策，写了设计规格文档，然后让 6 个 AI 审查者审了两轮，找出了 51 个问题全部修掉了。现在的设计是经过了充分推敲的。

二、系统怎么运作（5分钟）

我用最简单的话讲一下这个系统是怎么工作的。

你给系统一个目标，比如“分析这个内存镜像，找出可疑进程”。然后系统里的 **Supervisor（调度 Agent）** 会把这个大任务拆成小任务——先识别操作系统版本，再提取进程列表，再扫描隐藏进程，再分析网络连接——每个小任务派给专门的子 Agent 去做。最后 Supervisor 把所有结果汇总起来，生成一份带置信度标注的报告。

不是你手动敲命令，而是你下目标，系统自动拆、自动分、自动汇总。

整个设计有六条核心原则，记住三条最重要的就行：

第一条，抽象优先。 每个组件都有接口层，换底层实现不影响上层。比如 LLM 层，我们一套代码同时支持 OpenAI、Anthropic、DeepSeek、Ollama。线上比赛用云端模型，线下断网比赛用本地模型，设一个环境变量就切换了，Agent 代码一行不用改。

第二条，程序做搬运，AI 做判断。 文件同步、哈希校验、格式验证这些确定性的事情让程序做。AI 只负责理解内容、做策略判断、生成报告。AI 绝对不写网络调用来做协作——它只读写本地文件，文件同步交给 Syncthing。

第三条，置信度分级。 所有 AI 产出的东西必须标注：工具直接验证过的打 ，AI 推理出来但没验证的打 ，数据不够无法判断的打 。这样我们一眼就知道哪些结论可以直接用，哪些需要人再看一下。

技术栈：Python 3.12 写主体，性能敏感的地方用 Rust。数据分两条路径——实时协作走本地 YAML 文件，历史查询走 PostgreSQL。

三、团队怎么分工（5分钟）

我们不搞传统的部门制。用 **Cell 制**。

什么意思？每个人配一个 AI，你就是你这个 Cell 的老板，AI 是你的员工。你的任务不是写代码，是告诉 AI 要写什么代码，然后审查它写的对不对。

代码仓库这样分：

core 仓库——核心框架。这个我来亲自控，加上两位师兄帮忙。这是整个系统的核心，接口契约都在这里定义。

plugin-forensics——取证插件。两位打过比赛的同学来负责，你们对取证工具最熟。

plugin-pentest——渗透测试插件。一个老手带一个新手。

plugin-osint——开源情报插件。一个中级的带一个新手。

plugin-ctf——CTF 竞赛支持插件。负责比赛场景下的任务调度、计时管理、答案提交、题目类型识别等竞赛流程自动化。不是独立的安全领域，而是让我们的取证和渗透能力在比赛中高效运转的指挥模块。

knowledge-base——知识库。这个是自动化的，CI 会自动从各个插件仓库把知识汇入过来，不需要人维护。

我知道大家水平不一样，这没关系。有三种路径：

有基础的同学，直接当插件 owner，管自己的 AI 干活。有想法的同学，参与核心框架贡献和 Code Review。零基础的同学，从写测试和文档开始——测试是学代码最快的方式，你看测试用例就知道每个模块怎么用。而且每个模块我们都会写 TUTORIAL.md，零基础跟着走就能上手。最关键的是，你有 AI 当一对一私教。

四、第一个月做什么（10分钟）

整个项目分五个阶段，大概三个月出一个完整版本。第一个阶段三周，目标是搭好骨架，跑通第一条完整流程。

具体的任务有九个：

Task 1：项目脚手架。 就是建目录、写 pyproject.toml、配好开发环境。这个我今天就能搞定。

Task 2：LLM 抽象层。 这是整个系统最重要的基础设施——所有 Agent 都通过这个接口调大模型。我来做，加上一个师兄。

Task 3：OpenAI 和 Ollama 后端。 对接云端 API 和本地模型。这个技术含量不高，有 Python 基础加 AI 辅助就能做。

Task 4：DeepSeek、Anthropic 后端加路由。 DeepSeek 直接继承 OpenAI，Anthropic 需要独立写。再加一个 SensitivityRouter——高敏感任务自动走本地模型，低敏感走云端。谁有 API 对接经验可以来做。

Task 5：MessageBus 消息总线。 默认用 Python asyncio 进程内通信，零外部依赖。代码量很少，新手可以做。

Task 6：插件系统。 BasePlugin 抽象类、PluginManager 插件管理器、CapabilityRegistry 能力注册表。这个需要有面向对象基础的同学，要理解抽象类怎么用。

Task 7：数据库层。 PostgreSQL 建表、SQLAlchemy 模型、数据迁移。谁了解数据库的来做。

Task 8：CLI 命令行。 用 click 库写命令行工具。这个简单，新手可以做。

Task 9：端到端集成测试。 搭一条完整链路跑通。我和师兄来主导。

这些任务里，Task 2 必须先做，然后 3 和 4 接在后面。5、6、7、8 可以同时进行。我控 Task 1、2、9 三个关键节点。

本周的目标很简单：完成 Task 1 到 3，所有人把开发环境配好。

五、质量怎么保证（2分钟）

每个代码提交必须过四道门：

1. ruff 格式检查
2. mypy 类型检查
3. pytest 测试覆盖率不低于 70%
4. AI Code Review

这些都是 GitHub Actions 自动跑的，不是我来检查你。机器检查不伤感情。过不了就是过不了，改到过为止。零基础的同学不要怕，在报错中学习本来就是设计的一部分。我和师兄只看一件事：你的架构对不对，不是语法对不对。

六、协作用什么工具（2分钟）

GitHub 管代码——我们建一个 forhacker 的 GitHub Organization，所有仓库放在下面。

Syncthing 管协作数据——任务状态、每个人的发现、答案汇总，这些 YAML 文件通过 Syncthing 自动同步到所有人电脑上。不需要写任何网络代码，Syncthing 自己处理 NAT 穿透和断线重连，千万用户验证过的方案。

手机操控先用 Chrome Remote Desktop 过渡，后面我们自己开发一个 Bot 来做手机端通知和审批。

七、时间表（2分钟）

Phase 1：第 1-3 周，核心骨架和 LLM 层

Phase 2：第 4-6 周，Supervisor 调度引擎和第一个取证插件

Phase 3：第 7-9 周，团队协作功能、CI 流程、知识库自动汇入

Phase 4：第 10-12 周，MetaAgent 自进化系统、Web 可视化面板

八、今天回去要做的事

三件事:

第一, 装环境。 待会我把安装步骤发群里。需要装 uv (Python 包管理器)、Python 3.12、Ollama (本地模型)、Docker Desktop、Syncthing。今晚就装, 有问题群里问。

第二, 选方向。 三个核心领域: 电子取证、渗透测试、开源情报 (OSINT)。每个人选一个感兴趣的方向。另外, 对 CTF 竞赛流程自动化感兴趣的也可以选择做 CTF 插件。我会发一个接龙。

第三, 读文档。 我会把设计日志的 HTML 文件发群里, 浏览器打开就能看, 大概十分钟看完。想深入了解的去看设计规格的完整文档。

总结

我再强调一遍这次项目的核心理念:

我们不写代码, 我们指挥 AI 写代码。我们要建的是一套让多个 AI 协作取证的系统, 不是又搞一堆脚本。每个人管自己的 Cell, 核心组管接口契约。靠产出说话。

有什么问题现在提。